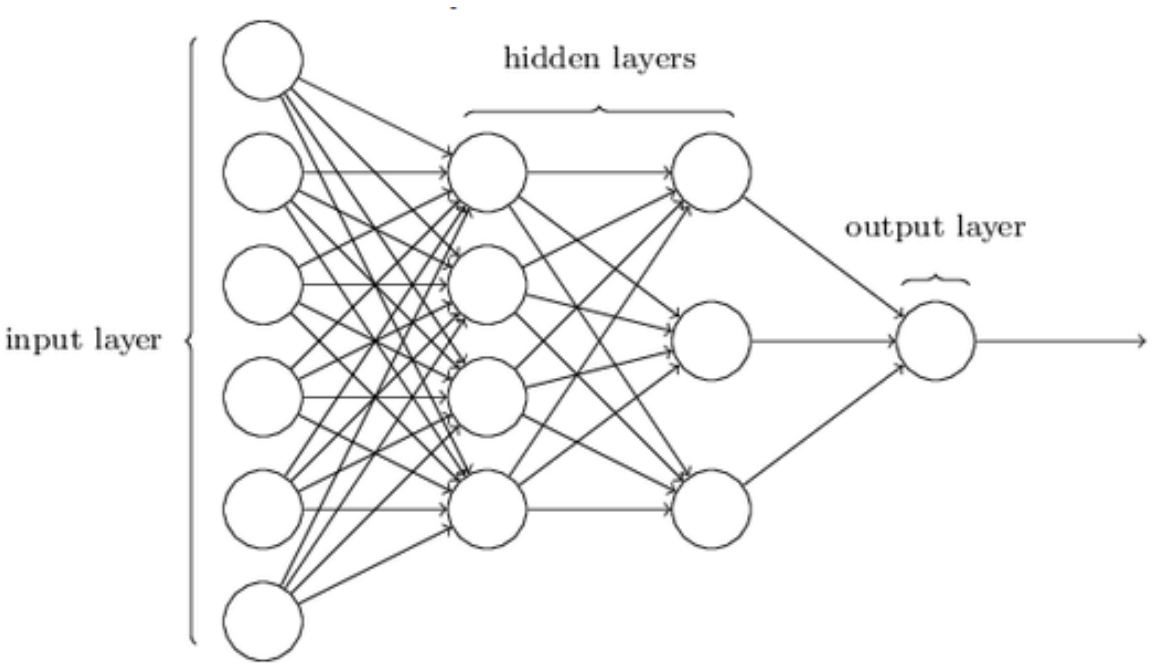
Deep Learning
W2: MLPs and backpropagation

M. Vazirgiannis

Ídistinguished Professor, Ecole Polytechnique, France
Google Scholar: https://bit.ly/2rwmvQU

September, 2026

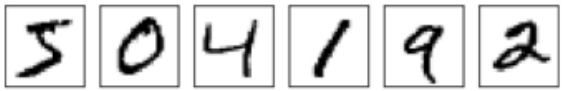
NNs architecture - Multilayer perceptron (MLP)

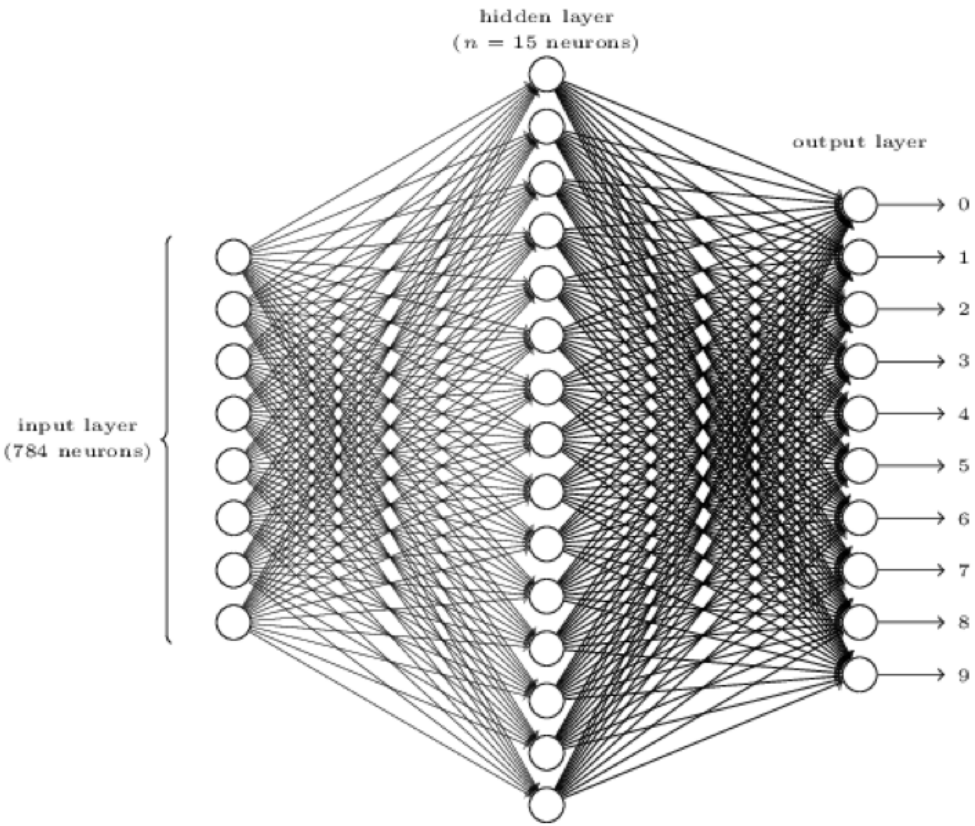


- i.e. case of 64x64 greyscale image representing a single number (i.e. "8")
- MLP with (64x64) 4096 input neurons
- A single neuron as output ($>$ 0.5: “is 8” , $<$ 0.5 not "8”)

Learning with gradient descent

- Assume hand written digits (MNIST database – 60.000 images):
- Each of them is a 28x28=784 pixels





- Assume a NN with 784 inputs, a hidden layer of 15 neurons and 10 outputs: i.e.: if x is an image (=7) $y(x)$: $(0,0,0,0,0,0,0,1,0,0)$ is the desired output.

http://neuralnetworksanddeeplearning.com/chap1.html

Learning with gradient descent

- Cost/Loss function $L(w,b) = \frac{1}{2n}\Sigma_x |y - f(x)|^2$
  - w: weights, b: biases, y: vector of correct outputs, n: total number of training samples
- Searching for w, b to minimize $L(w,b)$

$$w_k^{'} = w_k - \eta \frac{\partial L}{\partial w_k}$$

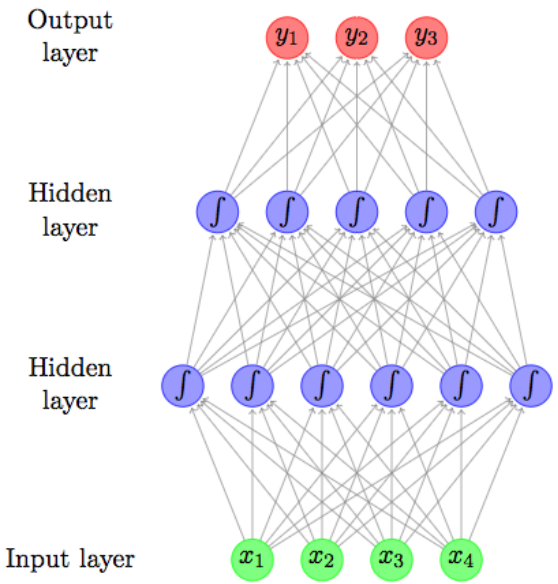
$$b_l^{'} = b_l - \eta \frac{\partial L}{\partial b_l}$$

Problems with GD: $L = \frac{1}{n}\Sigma_x L_x$ where $C_x = \frac{\|y-f(x)\|^2}{2}$

To compute the gradient ∇L we must compute gradients ∇L_x for each training point and then average them: $\nabla L = \frac{1}{n}\Sigma_x \nabla L_x$

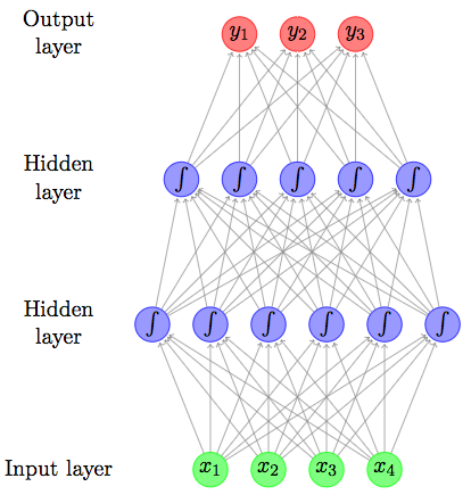
$\Rightarrow$ very slow learning => SGD...

Multilayer Feed-forward NNs



- top layer: output of the network (3 class problem).
- other layers are considered “hidden”.
- hidden layer neurons represent non-linear functions.
- applied to neuron’s value before passing it to the output.
- fully-connected: each neuron connected to all next layer neurons.

Multilayer Feed-forward NNs



- $\mathbf{x}$: input, $\mathbf{h}^{(i)}$: output of hidden layer i, $\mathbf{y}$: output

$$NN_{MLP2}(\mathbf{x}) = (g^2(g^1(\mathbf{x}\mathbf{W}^1 + \mathbf{b}^1)\mathbf{W}^2 + \mathbf{b}^2))\mathbf{W}^3$$

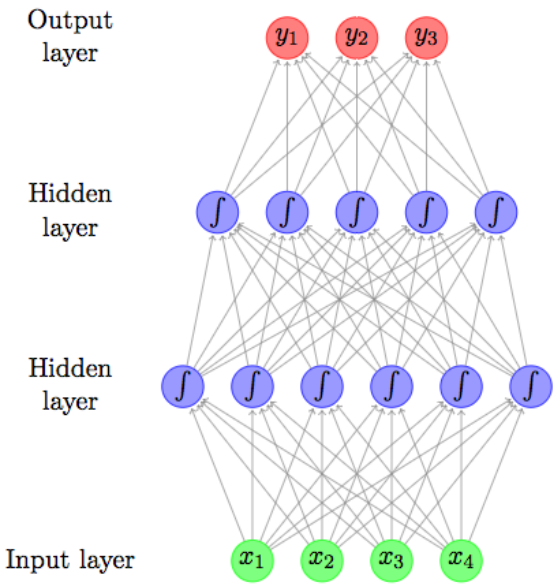
$\mathbf{x} \in \mathbb{R}^{d_{in}}, \mathbf{W}^1 \in \mathbb{R}^{d_{in} \times d_1}, \mathbf{b}^1 \in \mathbb{R}^{d_1}, \mathbf{W}^2 \in \mathbb{R}^{d_1 \times d_2}, \mathbf{b}^2 \in \mathbb{R}^{d_2}$,

- Alternatively:

$$\begin{aligned} NN_{MLP2}(\mathbf{x}) &= y \\ \mathbf{h}^1 &= g^1(\mathbf{x}\mathbf{W}^1 + \mathbf{b}^1) \\ \mathbf{h}^2 &= g^2(\mathbf{h}^1\mathbf{W}^2 + \mathbf{b}^2) \\ \mathbf{y} &= \mathbf{h}^2\mathbf{W}^3 \end{aligned}$$

- each $\mathbf{W}^i \in \mathbb{R}^{d_{in} \times d_{out}}$ represents embeddings of the data in d_{out} dimensions

Softmax for classification



$$y = NN_{MLP2}(\mathbf{x})$$
$$\mathbf{h}^1 = g^1(\mathbf{x}\mathbf{W}^1 + \mathbf{b}^1)$$
$$\mathbf{h}^2 = g^2(\mathbf{h}^1\mathbf{W}^2 + \mathbf{b}^2))$$
$$\mathbf{y} = \mathbf{h}^2\mathbf{W}^3$$

Classification: only one output activated - softmax:

$$\mathbf{y} = [y_1, \ldots, y_3]$$
$$softmax(y_i) = \frac{e^{y_i}}{\sum_{j=1}^{k} e^{y_j}}$$

- vector of non-negative real numbers that sum to 1
- Softmax transformation used when we need a probability distribution over the possible output classes.

Softmax

- Change the activation function in the output layer:

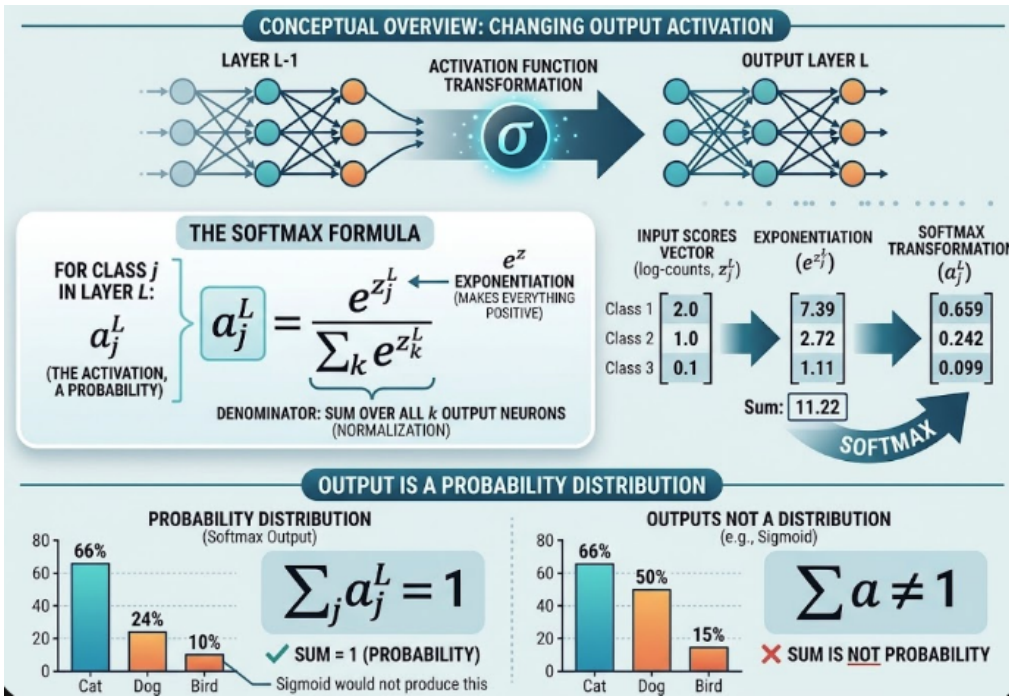
$$a_j^L = \frac{e^{z_j^L}}{\sum_k e^{z_k^L}}$$

  - Denominator: sum over all (k) output neurons

- Output is a probability distribution

$$\sum_j a_j^L = \frac{\sum_j e^{z_j^L}}{\sum_k e^{z_k^L}} = 1$$

  - sigmoid would not produce it



How do MLPs learn? - Backpropagation

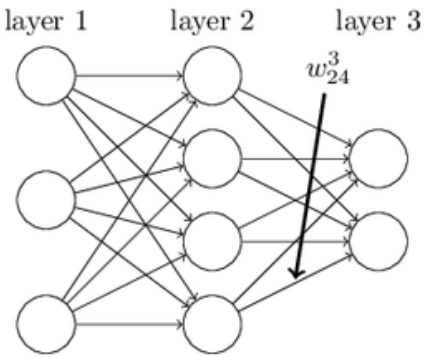
- Learning parameters with gradient descent

$$w_k^{'} = w_k - \eta \frac{\partial C}{\partial w_k}$$

$$b_l^{'} = b_l - \eta \frac{\partial C}{\partial b_l}$$

- How to compute the gradients: backpropagation

  w_{jk}^{l} : weight among the kth neuron of the $l-1$ layer to the jth neuron of the l layer.



Letters to Nature, Nature 323, 533-536 (9 October 1986) | doi:10.1038/323533a0; Learning representations by back-propagating errors, David E. Rumelhart*, Geoffrey E. Hinton† & Ronald J. Williams..

The backpropagation algorithm

- **Input x**: Set the corresponding activation α^l for the input layer.
- **Feedforward:** For each $l = 2, 3, \ldots, L$ compute $z^l = w^l \alpha^{l-1} + b^l$ and $\alpha^l = \sigma(z^l)$.
- **Output error** $delta^L$: Compute the vector $\delta^L = \nabla_a C \odot \sigma'(z^L)$
- **Backpropagate error**: For each $l = L-1, L-2, \ldots, 2$ compute

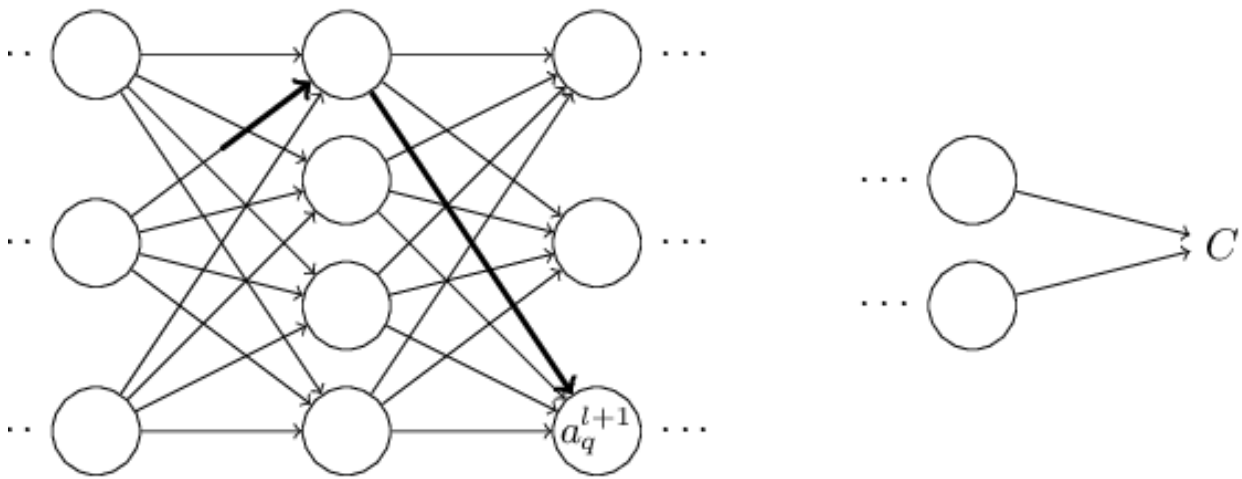
$$\delta^l = ((w^{l+1})^T \sigma^{l+1}) \odot \sigma'(z^L).$$

- **Output**: gradient of the cost function:

$$\partial C / \partial w^l_{jk} = a^{l-1}_k \delta^l_j$$
$$\partial C / \partial b^l_j = \delta^l_j$$

Back propagation - the big picture

- Δw_{lj}^k : small change in weight w_{lj}^k
- causes change of the output activation from the corresponding neuron: $\Delta \alpha_j^l \approx \frac{\partial \alpha_j^l}{\partial w_{jk}^l} \Delta w_{jk}^l$
- causing changes in all activations in the next layer
- Thus: $\Delta \alpha_q^{l+1} \approx \frac{\partial \alpha_q^{l+1}}{\partial \alpha_j^l} \Delta \alpha_j^l$

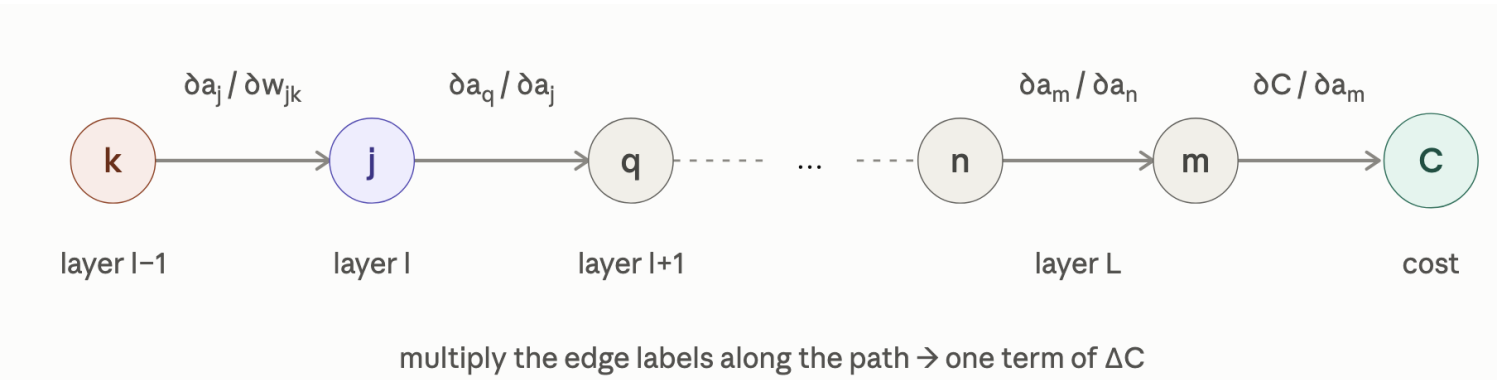


Back propagation - the big picture

Cascading changes to next layers cause change in the output:

- Single path:

$$\Delta C \approx \frac{\partial C}{\partial a_m^L} \frac{\partial a_m^L}{\partial a_n^{L-1}} \frac{\partial a_n^{L-1}}{\partial a_p^{L-2}} \cdots \frac{\partial a_q^{l+1}}{\partial a_j^l} \frac{\partial a_j^l}{\partial w_{jk}^l} \Delta w_{jk}^l$$

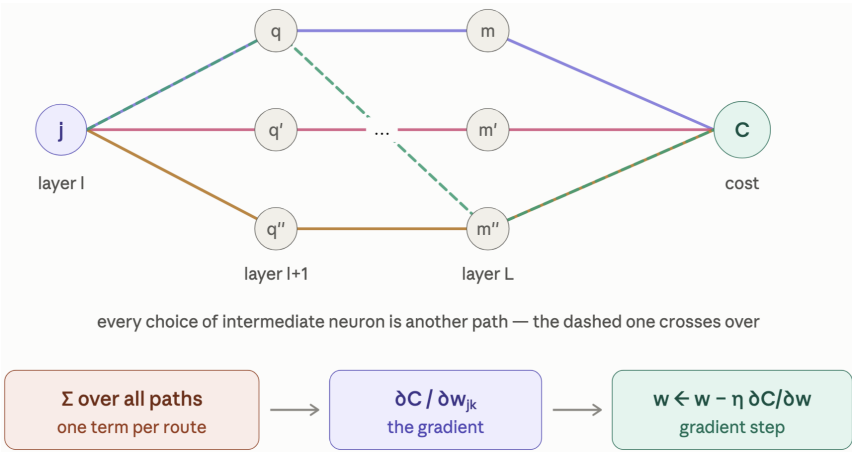


Back propagation – the big picture

Cascading changes to next layers cause change in the output:

- **Sum over all paths:**

$$\Delta C \approx \sum_{mnp\ldots q} \frac{\partial C}{\partial a_m^L} \frac{\partial a_m^L}{\partial a_n^{L-1}} \frac{\partial a_n^{L-1}}{\partial a_p^{L-2}} \cdots \frac{\partial a_q^{l+1}}{\partial a_j^l} \frac{\partial a_j^l}{\partial w_{jk}^l} \Delta w_{jk}^l$$



- **Effect of weight change on total cost:**

$$\frac{\partial C}{\partial w_{jk}^l} \approx \sum_{mnp\ldots q} \frac{\partial C}{\partial a_m^L} \frac{\partial a_m^L}{\partial a_n^{L-1}} \frac{\partial a_n^{L-1}}{\partial a_p^{L-2}} \cdots \frac{\partial a_q^{l+1}}{\partial a_j^l} \frac{\partial a_j^l}{\partial w_{jk}^l}$$

- **Weights update:**

$$w_k' = w_k - \eta \frac{\partial C}{\partial w_k}$$

$$b_l' = b_l - \eta \frac{\partial C}{\partial b_l}$$

1. Computation graphs — motivation

- A layer is *not* atomic. `ReLU(Wx+b)`: matrix multiply, an addition, and an elementwise max — three primitives we can each differentiate.
- Assume a **directed acyclic graph** - nodes are primitive operations, edges carry tensors/values.
- Forward pass is evaluating nodes in topological order. Loss is the unique sink - last node.

- The graph is **data, not code** — built dynamically (PyTorch) or compiled ahead of time (JAX). Either way can traversed later.

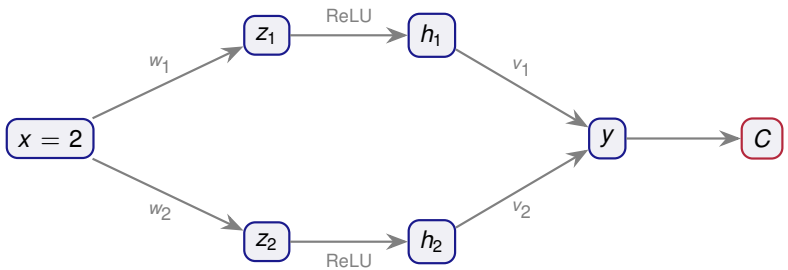
"Deep learning frameworks" are differentiable...

Computation Graph Example — forward pass by hand

$$z_1 = w_1 x + b_1, \quad h_1 = \mathrm{ReLU}(z_1)$$
$$z_2 = w_2 x + b_2, \quad h_2 = \mathrm{ReLU}(z_2)$$
$$y = v_1 h_1 + v_2 h_2, \quad C = \tfrac{1}{2}(y - t)^2$$

x	t	w_1	b_1	w_2	b_2	v_1	v_2
2	3	0.5	-0.5	-1	3	2	3

Node	Expression	Value
z_1	$0.5 \cdot 2 - 0.5$	**0.5**
h_1	$\max(0, 0.5)$	**0.5**
z_2	$-1 \cdot 2 + 3$	**1.0**
h_2	$\max(0, 1.0)$	**1.0**
y	$2(0.5) + 3(1.0)$	**4.0**
C	$\tfrac{1}{2}(4 - 3)^2$	**0.5**



- How many nodes if you count *every* primitive?
- Which values must be **stored** for the backward pass? *h_1, h_2 and the signs of z_1, z_2 — one bit per ReLU unit. First encounter with the memory cost of training.*

Multivariable Chain Rule: Tracing the Paths

The input x affects the final cost $C = \frac{1}{2}(y - t)^2$ through two parallel hidden branches ($z_1 \to h_1$ and $z_2 \to h_2$):

$$\frac{\partial C}{\partial x} = \underbrace{\frac{\partial C}{\partial y} \cdot \frac{\partial y}{\partial h_1} \cdot \frac{\partial h_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial x}}_{\text{Path A (Top Branch)}} + \underbrace{\frac{\partial C}{\partial y} \cdot \frac{\partial y}{\partial h_2} \cdot \frac{\partial h_2}{\partial z_2} \cdot \frac{\partial z_2}{\partial x}}_{\text{Path B (Bottom Branch)}}$$

Step-by-Step Evaluation:

- **Common Output Term:** $\frac{\partial C}{\partial y} = y - t = 4.0 - 3.0 = \mathbf{1.0}$
- **Path A ($x \to z_1 \to h_1 \to y \to C$):**

$$\text{Path A} = \underbrace{1.0}_{\partial C/\partial y} \cdot \underbrace{2.0}_{v_1} \cdot \underbrace{1.0}_{\text{ReLU}'(0.5)} \cdot \underbrace{0.5}_{w_1} = +\mathbf{1.0}$$

- **Path B ($x \to z_2 \to h_2 \to y \to C$):**

$$\text{Path B} = ??$$

Automatic Differentiation: Principles & Adjoints

Path Aggregation and Directional Importance

Pathway	Local Chain Product	Gradient Value
Path A ($x \to z_1 \to h_1 \to y \to C$)	$1.0 \times 2.0 \times 1.0 \times 0.5$	$+\mathbf{1.0}$
Path B ($x \to z_2 \to h_2 \to y \to C$)	$1.0 \times 3.0 \times 1.0 \times (-1.0)$	$-\mathbf{3.0}$
Total Gradient ($\frac{\partial C}{\partial x} = \sum$ Paths)	$(+1.0) + (-3.0)$	$-\mathbf{2.0}$

Key Insights:

- **Opposite Signs:** Path A pushes positively ($+1.0$), whereas Path B pulls negatively (-3.0).
- **Directional Sensitivity:** Inspecting Path A alone gives the **wrong gradient sign** ($+1.0$ vs -2.0).
- **Accumulation Principle:** Reverse-mode autodiff guarantees exact updates by accumulating gradients across **all active computational branches** at converging nodes.

Why Reverse-Mode AD Powers Deep Learning

Autodiff computes exact numerical derivatives without symbolic expansion or finite differences:

- **Not Symbolic:** Avoids exponential formula growth ("expression swell").
- **Exact & Composable:** Evaluates derivatives numerically using known local primitive rules.

The Core Complexity Asymmetry:

Method	**Sweep Direction**	**Cost for P Params / 1 Loss**	**Ideal Domain**
Forward-Mode AD	Inputs $\rightarrow$ Outputs	P sweeps	Few inputs, many outputs
Reverse-Mode AD	**Output $\rightarrow$ Inputs**	**1 sweep**	**Many inputs, scalar loss**

Deep learning maps P parameters to 1 *scalar loss* $\Longrightarrow$ *Backpropagation is reverse-mode AD on a scalar loss.*

The Reverse Sweep: Adjoint Propagation

During the backward pass, we traverse nodes in **reverse topological order**, computing one **adjoint** $\bar{v}$ per node:

$$\bar{v} \triangleq \frac{\partial C}{\partial v}$$

Step 1: Backward Propagation through Activations

- **Loss seed:** $\bar{C} = \frac{\partial C}{\partial C} = 1.0$
- **Output adjoint:** $\bar{y} = \frac{\partial C}{\partial y} = y - t = 4.0 - 3.0 = \mathbf{1.0}$
- **Post-activation adjoints:**
  $\bar{h}_1 = \bar{y} \cdot v_1 = 1.0 \times 2 = \mathbf{2.0}, \quad \bar{h}_2 = \bar{y} \cdot v_2 = 1.0 \times 3 = \mathbf{3.0}$
- **Pre-activation adjoints:**

$$\bar{z}_1 = \bar{h}_1 \cdot \mathbf{1}_{\{z_1 > 0\}} = 2.0 \times 1 = \mathbf{2.0}, \qquad \bar{z}_2 = \bar{h}_2 \cdot \mathbf{1}_{\{z_2 > 0\}} = 3.0 \times 1 = \mathbf{3.0}$$

Parameter Gradients and Dynamic Programming

Step 2: Single-Sweep Parameter Gradients

Parameter	Adjoint Expression	Substitution	Gradient
v_1, v_2	$\bar{v}_1 = \bar{y} \cdot h_1, \quad \bar{v}_2 = \bar{y} \cdot h_2$	$1.0 \times 0.5, \quad 1.0 \times 1.0$	$\bar{v}_1 = \mathbf{0.5}, \ \bar{v}_2 = \mathbf{1.0}$
w_1, b_1	$\bar{w}_1 = \bar{z}_1 \cdot x, \quad \bar{b}_1 = \bar{z}_1 \cdot 1$	$2.0 \times 2, \quad 2.0 \times 1$	$\bar{w}_1 = \mathbf{4.0}, \ \bar{b}_1 = \mathbf{2.0}$
w_2, b_2	$\bar{w}_2 = \bar{z}_2 \cdot x, \quad \bar{b}_2 = \bar{z}_2 \cdot 1$	$3.0 \times 2, \quad 3.0 \times 1$	$\bar{w}_2 = \mathbf{6.0}, \ \bar{b}_2 = \mathbf{3.0}$

Key System Consequences:

- **Memoization:** The node adjoint $\bar{z}_1 = 2.0$ is computed **once** and reused to update both w_1 and b_1, avoiding redundant path multiplications.
- **Memory Footprint:** Intermediate forward values (h_1, h_2, x) must remain in memory until consumed during the backward pass (the driver for activation checkpointing).

3. Autodiff — the reverse sweep as a table

Visit nodes in reverse topological order, carrying one **adjoint** per node, $\bar{v} = \partial C / \partial v$.

Node	Adjoint rule	Value
C	seed	1
y	$y - t$	**1.0**
h_1	$\bar{y}\, v_1$	**2.0**
h_2	$\bar{y}\, v_2$	**3.0**
z_1	$\bar{h}_1\, \mathbf{1}[z_1 > 0]$	**2.0**
z_2	$\bar{h}_2\, \mathbf{1}[z_2 > 0]$	**3.0**
$\mathbf{v_1}$	$\bar{y}\, h_1$	**0.5**
$\mathbf{v_2}$	$\bar{y}\, h_2$	**1.0**
$\mathbf{w_1}$	$\bar{z}_1 x$	**4.0**
$\mathbf{b_1}$	$\bar{z}_1$	**2.0**
$\mathbf{w_2}$	$\bar{z}_2 x$	**6.0**
$\mathbf{b_2}$	$\bar{z}_2$	**3.0**

Six gradients, **one** sweep.

Method	Here	P params
Finite differences	12	$2P$ passes
Forward-mode AD	6	P sweeps
Reverse-mode AD	**1**	**1** sweep

For a 7B-parameter model the first two rows are not slow — they are impossible.

Note $\bar{h}_1 = \bar{y} v_1 = 2.0$ is computed *once* and reused by everything upstream. In a wide network that single reused number replaces an exponential set of path products.

Two consequences: activations stay in memory until consumed (checkpointing trades recompute for memory); any non-differentiable op breaks the chain.

What Does "Differentiable" Mean?

A deep learning framework is **differentiable** because it automatically evaluates gradients without manual analytic derivations.

The Autodiff Mechanism:

- **Not Symbolic:** Does not manipulate algebraic expressions (e.g., Mathematica).
- **Not Numerical:** Does not approximate via finite differences ($\frac{f(x+\epsilon)-f(x)}{\epsilon}$).
- **Automatic Differentiation:** Evaluates **exact derivatives** by decomposing computations into elementary operations and applying the chain rule step-by-step.

The Node Contract: Every registered primitive operation implements a dual interface:

1. **Forward Pass:** Computes output values and caches inputs (memoization).
2. **Backward Pass:** Computes vector-Jacobian products (local gradients).

The Computation Graph & Primitive Primitives

During the forward pass, the runtime builds a directed acyclic graph (DAG). The backward pass traverses this graph in reverse order using the product rule.

Operation	**Forward Function**	**Backward Derivative**
Addition	$c = a + b$	$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial c}, \quad \frac{\partial L}{\partial b} = \frac{\partial L}{\partial c}$
Multiplication	$c = a \cdot b$	$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial c} \cdot b, \quad \frac{\partial L}{\partial b} = \frac{\partial L}{\partial c} \cdot a$
Sigmoid	$y = \sigma(x)$	$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \sigma(x)(1 - \sigma(x))$
ReLU	$y = \max(0, x)$	$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \mathbf{1}_{\{x>0\}}$
Matrix Multiply	$Y = WX$	$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial Y} X^\top, \quad \frac{\partial L}{\partial X} = W^\top \frac{\partial L}{\partial Y}$

Gradient flow — motivation

Gradient flow: "journey" of error signal traveling backward through a neural network during training - gradient reaching an early layer is a **product of many Jacobians/derivatives**. Products (i.e. *factor*) may behave badly:

- typical product $< 1 \Rightarrow$ geometric decay $\Rightarrow$ vanishing, early layers stop learning;
- typical product $> 1 \Rightarrow$ geometric growth $\Rightarrow$ exploding, loss goes to NaN.

How to tackle this problem:

- **Sigmoid/tanh**: derivatives $0.25 - 1 \leq 1 \Rightarrow$ vanishing gradients in early layers
- **ReLU**: derivative exactly 1 when active: prevents gradient shrinking when neuron is active thus easier train deeper networks.(cost: dead units).
- **Residual connections**: add input of a layer directly to its output (x+F(x)). During backpropagation gradient can bypass layers entirely (adding a derivative of 1), preventing it from vanishing.
- **Normalization**:(e.g., Batch/Layer Norm): keep activations/gradients within a stable scale, preventing exploding/vanishing.
- **Gradient clipping**: gradients capped at a maximum value - prevent "exploding gradient".

Architecture design: practice of keeping a product of Jacobians close to 1.

Gradient flow — the arithmetic, then the measurement

(a) Powers of a single factor. f^L: f:derivative activation function, L: depth of network, for sigmoid: $f = 0.25$, $tanh = 0.42$,

Depth L	0.25^L (sigmoid, *best* case)	0.42^L (tanh at $\lvert z\rvert \approx 1$)	1.5^L (weights too large)
1	2.5×10^{-1}	4.2×10^{-1}	1.5
5	9.8×10^{-4}	1.3×10^{-2}	7.6
10	9.5×10^{-7}	1.7×10^{-4}	5.8×10^{1}
20	9.1×10^{-13}	2.9×10^{-8}	3.3×10^{3}

Sigmoid's *best possible* factor still kills the signal by 10^6 over ten layers. A factor of exactly 1 — which is what ReLU (active side) and a residual identity branch provide.

(b) Measured: 10-layer MLP, width 64, batch 256, $\|\partial C/\partial W_l\|$.

Layer	1	2	3	4	5	6	7	8	9	10
Sigmoid	8.8e−7	3.1e−5	1.3e−4	6.3e−4	3.2e−3	1.2e−2	4.8e−2	2.3e−1	9.7e−1	4.2
ReLU	3.5e1	1.1e2	1.6e2	1.7e2	1.8e2	2.2e2	1.7e2	1.5e2	1.2e2	1.0e2

First layer / last layer: 2×10^{-7} for sigmoid, 0.35 for ReLU. With any single learning rate, sigmoid layer 1 is frozen while layer 10 trains.

Weight Initialization: tanh Networks

Setup: 10-layer network, layer width $n = 128$, weights initialized as $W \sim \mathcal{N}(0, \sigma^2)$.

Initialization (σ)	L1	L2	L3	L5	L7	L10
0.01 *(too small)*	0.111	0.013	1.4e−3	1.8e−5	2.4e−7	3.4e−10
1.0 *(too large)*	0.964	0.964	0.963	0.963	0.963	0.963
$\sqrt{1/n}$ *(Xavier)*	0.628	0.485	0.405	0.318	0.272	0.223

Cell values: *empirical standard deviation* std($a^{(l)}$) of post-activation values at layer l. A healthy network requires standard deviations to remain non-zero and stable without saturating the non-linearity.

- $\sigma = 0.01$ **(Vanishing Activations):** Signal magnitude decays exponentially toward zero (3.4×10^{-10}).
- $\sigma = 1.0$ **(The Saturation Trap):** std ≈ 0.96 looks stable, but outputs are pinned to the flat tails (± 1) where $\tanh'(z) \approx 0$, completely killing backward gradient flow.
- $\sigma = \sqrt{1/n}$ **(Xavier / Glorot):** Preserves variance within the active linear regime ($0.628 \rightarrow 0.223$) without saturation.

Weight Initialization: ReLU Networks

Setup: 10-layer network, layer width $n = 128$, weights initialized as $W \sim \mathcal{N}(0, \sigma^2)$.

Initialization (σ)	L1	L2	L3	L5	L7	L10
$\sqrt{1/n}$ *(Xavier)*	0.586	0.411	0.293	0.160	0.080	0.028
$\sqrt{2/n}$ *(He / Kaiming)*	0.826	0.795	0.766	0.735	0.646	0.676

cell values:

empirical standard deviation std($a^{(l)}$) of the ReLU activations ($\max(0, z)$) at layer l.

- **Xavier fails on ReLU (**$0.586 \to 0.028$**):** assumes zero-mean, symmetric activations. Because $\text{ReLU}(z) = \max(0, z)$ eliminates ñalf incoming inputs, variance is halved at each successive layer:

$$\text{Var}[\text{ReLU}(z)] \approx \frac{1}{2}\text{Var}[z]$$

- **He / Kaiming succeeds (**$0.826 \to 0.676$**):** Setting $\sigma = \sqrt{2/n}$ (variance $2/n$) scales the pre-activations by a factor of 2, exactly neutralizing the 50% variance loss discarded by ReLU.

Activation Functions — Motivation

An activation function $f(z)$ is applied to a neuron's linear output $z = w^T x + b$.

Why are they critical?

- **Non-linearity:** Without $f(z)$, stacking multiple layers just yields a linear combination: $W_2(W_1 x) = (W_2 W_1)x$. The network could not learn complex, ill-conditioned loss landscapes.
- **Gradient Flow:** During backpropagation, the gradient ∇L is multiplied by the derivative of the activation function $f'(z)$ at every layer.
- **The Jacobian Product:** For a network of depth L, the gradient reaching the early layers is proportional to the product of derivatives:

$$\nabla L_{\text{early}} \propto \prod_{i=1}^{L} f'(z_i)$$

If $f'(z)$ is consistently less than 1, this product vanishes. If it is greater than 1, it explodes. The choice of $f(z)$ dictates whether the optimizer receives a usable gradient.

The Problem: Sigmoid and Vanishing Gradients

The Sigmoid function squashes inputs into a range between 0 and 1.

Equations:

$$f(z) = \frac{1}{1 + e^{-z}}$$
$$f'(z) = f(z)(1 - f(z))$$

The Arithmetic of Failure:

- The maximum value of $f'(z)$ is **0.25** (when $z = 0$). For most other inputs, it is much smaller.
- As seen previously, the Jacobian product for a depth $L = 20$ network is $(0.25)^{20} \approx 10^{-12}$.

Impact on the Optimizer: The update rule $w_t = w_{t-1} - \eta \nabla L$ fails. The gradient ∇L reaching the early layers is so close to zero that the weights effectively freeze. Momentum and Adam cannot fix this, because there is no gradient signal to accumulate or adapt to.

The Solution: ReLU and Gradient Highways

The Rectified Linear Unit (ReLU) solves the vanishing gradient problem by acting as a "gradient highway".

Equations:

$$f(z) = \max(0, z)$$

$$f'(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$

The Arithmetic of Success:

- For active neurons ($z > 0$), the derivative is exactly **1.0**.
- The Jacobian product becomes $1^L = 1$. The gradient flows backward perfectly unchanged, regardless of depth L.

Impact on the Optimizer: Because ∇L arrives at the early layers at full scale, our optimizers work as intended. A standard update $w_t = w_{t-1} - \eta \nabla L$ takes a proper step. Adam's scaling $\eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$ functions correctly because $\hat{m}_t$ and $\hat{v}_t$ accurately reflect the true, unscaled loss landscape.

Optimization in Deep Learning - Mathematical Formulations

Gradient Descent, Momentum, and Adam Dynamics Given stochastic gradient $g_t = \nabla_\theta \mathcal{L}(\theta_t)$ and learning rate η:

1. SGD with Classical Momentum (coefficient $\beta \in [0, 1)$) - usually $\beta = 0.99$

$$v_t = \beta v_{t-1} + (1 - \beta) g_t, \qquad \theta_{t+1} = \theta_t - \eta v_t$$

Accumulates velocity along persistent directions; Gradients pointing in consistent directions add-up/accelerate, orthogonal oscillating gradients cancel each other.

2. Adam (Adaptive Moment Estimation)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \qquad \hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad (\beta_1 = 0.9)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (\beta_2 = 0.999)$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \odot \hat{m}_t \qquad (\epsilon \approx 10^{-8})$$

Combines first-moment velocity with second-moment coordinate-wise variance scaling.

Example: An Ill-Conditioned Bowl

Consider the **loss function:** $L(w_1, w_2) = w_1^2 + 10w_2^2$

- w_1 **(Flat direction):** Gradient is small. $\nabla L_{w_1} = 2w_1$
- w_2 **(Steep direction):** Gradient is 10x larger. $\nabla L_{w_2} = 20w_2$

The Problem: Basic SGD uses a small learning rate to avoid exploding in the steep w_2 direction. This causes painfully slow progress in the flat w_1 direction, resulting in a violent zig-zag path.

The Challenge: Anisotropic Curvature

Consider the quadratic bowl/ravine **loss function** $L(w_1, w_2) = w_1^2 + 10w_2^2$ with condition number $\kappa = \frac{\lambda_{\max}}{\lambda_{\min}} = 20$ where λ derivative of the relevant dimension

- Gradient Descent Update:

$$\theta_t = \theta_{t-1} - \eta \nabla L(w_{t-1})$$

- Applied to our curve ($\eta = 0.1$): $\theta_{1,t} = \theta_{1,t-1} - 0.1 \cdot (2\theta_{1,t-1})$; $\theta_{2,t} = \theta_{2,t-1} - 0.1 \cdot (20\theta_{2,t-1})$
- start at ($\theta_1 : w_1 = 2, \theta_2 : w_2 = 2$): θ_1 gradient is 4; θ_2 gradient is 40.

Step t	x **(**$\lambda = 1$**)**	y **(**$\lambda = 20$**)**	$f(x, y)$	**Dynamics (lr** $= 0.02$**)**
0	$+5.0000$	$+1.0000$	22.500	Initial point
1	$+4.9000$	$+0.6000$	15.605	y drops 40%
2	$+4.8020$	$+0.3600$	12.826	y drops 64%
4	$+4.6118$	$+0.1296$	10.803	y -87%, x only -8%

- **Single Learning Rate Dilemma:** A step size suitable for x causes catastrophic divergence in y; a step size safe for y stalls progress along x.
- **Stability Bound:** Standard gradient descent diverges if $\eta \geq \frac{2}{\lambda_{\max}} = \frac{2}{20} = 0.1$.
- At $\eta = 0.09$, plain SGD converges slowly ($f \to 0.2$ in 20 steps); at $\eta = 0.11$, it explodes ($f \to 384.6$).

Momentum: The Fix for Zig-Zagging

Momentum adds a fraction ($\beta = 0.9$) of the previous step to the current step.
Equations:

$$v_t = \beta v_{t-1} + \eta \nabla L(w_{t-1})$$
$$w_t = w_{t-1} - v_t$$

Applied to our curve ($\eta = 0.1$):

- **In w_2 (steep):** optimizer overshoots (high value), but on the next step, the gradient reverses. The old velocity and new gradient **cancel out**, dampening the zig-zag.
- **In w_1 (flat):** The gradient never changes direction. Velocity compounds ($0.1 \rightarrow 0.19 \rightarrow 0.27$), **accelerating** down the flat valley floor.

Adam: The Ultimate Fix for Ill-Conditioning

Adam calculates moving averages of the gradient (m_t) and squared gradient (v_t).
Let's use $\eta = 0.1$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$.

- **First Moment (Momentum):** $m_t = \beta_1 m_{t-1} + (1 - \beta_1)\nabla L(w_{t-1})$
- **Second Moment (Squared Gradient):** $v_t = \beta_2 v_{t-1} + (1 - \beta_2)(\nabla L(w_{t-1}))^2$
- **Bias Correction (crucial for early steps):** $\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$
- **The Update Step:** $w_t = w_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\epsilon}$

Applied to curve (considering two directions independently):

- **For w_1 (flat):** small gradient (≈ 4). $\hat{m}_{1,t}$ small (≈ 4). The squared gradient $\hat{v}_{1,t}$ also small (≈ 16).
  Effective step $\approx \eta \frac{4}{\sqrt{16}} = \eta \frac{4}{4} = \eta$.
- **For w_2 (steep):** huge gradient (≈ 40). $\hat{m}_{2,t}$ huge (≈ 40). squared gradient $\hat{v}_{2,t}$ massive (≈ 1600).
  Effective step $\approx \eta \frac{40}{\sqrt{1600}} = \eta \frac{40}{40} = \eta$.

dividing by $\sqrt{v_t}$, Adam **neutralizes the scale** of the gradient - reshapes ill-conditioned bowl into a perfect circle, walking straight to the minimum without zig-zagging.

Comparison & Practical Guidelines

Optimizer	Core Strengths	Practical Limitations
SGD + Momentum	Strong generalization in vision; lower memory footprint (1 state/param).	Highly sensitive to η; slow in ill-conditioned valleys without fine-tuning.
Adam / AdamW	Invariant to gradient scale; robust to sparse gradients; standard for LLMs.	Higher memory (2 states/-param); prone to overshooting without decay schedules.

Key Takeaways for Practice:

- **Minibatch Noise is Functional:** Stochastic gradient noise prevents premature convergence in sharp, poorly-generalizing local minima.
- **Schedules are Mandatory for Adam:** Because Adam normalizes gradient magnitude, an explicit learning rate schedule (warmup followed by cosine/linear decay) is strictly required to force parameter convergence.

Closing: the pipeline in one sentence

Build the graph $\rightarrow$ evaluate it forward $\rightarrow$ sweep back once with reverse-mode AD, reusing shared sub-results $\rightarrow$ obtain every parameter's sensitivity $\rightarrow$ convert those into a step with an optimizer $\rightarrow$ repeat.

Initialization sets where the loop begins. Architecture determines whether the backward signal survives the trip. The optimizer determines how the information is spent.

Symptom	**Likely cause**	**Topic**
Loss is NaN after a few steps	Exploding gradients, lr too high	Flow, optimizer
Loss flat from step one	Dead units, bad init scale, lr too low	Init, flow
Early layers barely change	Vanishing gradients	Flow
Loss oscillates without improving	lr too high, no momentum	Optimizer
Trains then diverges suddenly	No warmup, bad schedule	Schedule
OOM in backward but not forward	Stored activations	Autodiff
Gradient exactly zero where it should not be	Non-differentiable op	Graph

Backpropagation Quiz

1. What is the primary purpose of backpropagation in neural networks?

A) To initialize weights and biases

B) To update weights based on errors

C) To add more neurons to the network

D) To increase the number of layers in the network

2. In backpropagation, which function is used to measure the difference between the predicted and actual outputs?

A) Activation function

B) Loss function

C) Optimization function

D) Regularization function

3. What is the role of the gradient in backpropagation?

A) It decides the activation function

B) It helps adjust the weights to minimize the loss

C) It adds random values to weights and biases

D) It resets weights and biases to initial values

4. In which order does backpropagation update the layers of a neural network?

A) From input to output layer

B) Randomly across all layers

C) From output to input layer

D) It only updates the output layer

6. What is the vanishing gradient problem in backpropagation?

A) Gradients become too large and cause instability

B) Gradients become very small and slow down learning

C) Gradients reverse direction during backpropagation

D) Gradients stop at the output layer and don't propagate

7. Which technique can help mitigate the vanishing gradient problem?

A) Using sigmoid activation function

B) Adding more layers to the network

C) Using activation functions like ReLU

D) Increasing learning rate